

Program 2: Regression Model

Quan Do^a^aCSEN - qldo@scu.edu

1. Introduction

In this project, I develop a regression model aimed at predicting housing prices using a comprehensive dataset. The objective is to apply regression methods on structured features derived from real estate data, including both numerical and categorical variables. Our workflow begins with data cleaning and exploratory data analysis to understand feature distributions, detect outliers, and identify missing values. We then proceed with feature engineering, encoding strategies, and normalization to prepare the data for modeling. Dimensionality reduction technique, such as PCA, is also explored to enhance model performance and reduce overfitting. The final model is trained, validated, and tested, with results visualized through prediction distribution plots and saved for further use. This end-to-end pipeline illustrates the practical application of machine learning for real-world regression tasks.

2. Libraries and Environment Setup

The codebase relies on several core Python libraries. `pandas` and `numpy` are used for data manipulation and numerical operations, respectively. `matplotlib.pyplot` and `seaborn` support visualization tasks, including plotting predicted price distributions. For preprocessing, `scikit-learn` provides tools such as `StandardScaler` and `OneHotEncoder` enabling streamlined transformation of numerical and categorical data. Dimensionality reduction is explored using PCA. For the regression models, I will be using Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regressor and XGB Regressor to make predictions and evaluate their performances. Auxiliary libraries such as `ast` and `re` assist in parsing and cleaning string-based features like amenities.

3. Data Exploration and Analysis

The training dataset has a total of 89524 samples and 66 features. On the other hand the testing dataset has a total of 22382 samples and 65 features (excluding 'price')

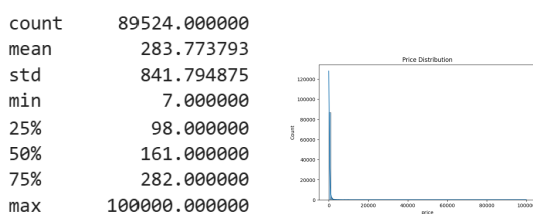


Figure 1. Price Distribution of Training Dataset.

Figure 1 illustrates the summary statistics and distribution of the price variable in the training dataset. The distribution is visibly right-skewed, indicating the presence of high-value outliers that may distort the model's learning. To mitigate the influence of these extreme values, the maximum price was capped at the 99th percentile, effectively reducing the impact of outliers on the regression model.

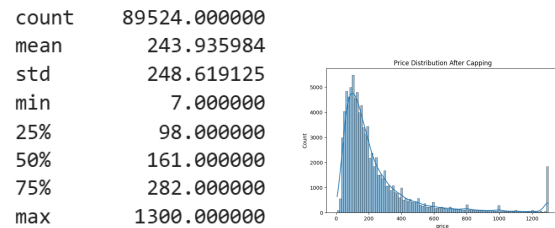


Figure 2. Price Distribution of Training Dataset.

Figure 2 presents the distribution of the price variable in the training dataset after capping extreme values at the 99th percentile. As shown, the distribution is right-skewed, with a long tail extending toward higher price ranges. The capping process limits the maximum price to \$1,300, which effectively reduces the influence of extreme outliers. This transformation helps stabilize the distribution and improves the model's ability to learn meaningful patterns without being disproportionately affected by rare, high-priced listings. The summary statistics indicate that the median price is \$161, with an interquartile range from \$98 to \$282, and a mean of approximately \$244.

3.1. Data Cleaning and Features Extraction

As an initial step in feature selection, any feature with more than 90% missing values was removed from the dataset. This threshold-based filtering helps eliminate irrelevant or uninformative features that are unlikely to contribute meaningfully to model performance.

3.1.1. Handling Non-Numerical Values

In this section, a manual review of all non-numeric features is conducted to classify them into one of four types: binary, categorical, free-text, or numeric values expressed as strings. Binary variables, typically represented as True or False, are encoded into binary format by mapping True to 1 and False to 0. Any missing values (NaN) in these binary columns are also imputed with 0 to denote the absence of the attribute. For features containing numeric information embedded within text, all non-digit characters are removed to extract and retain only the numerical values. Categorical variables are label-encoded by assigning each unique category an integer value from 1 to n, where n is the total number of distinct categories. Finally, unstructured free-text fields—such as description or name—are excluded from the analysis due to their high dimensionality and limited predictive value in the current modeling scope. At the end of this process, all of the non-numerical features will now be considered numerical values.

The only features that remain non-numerical that haven't been dropped are 'amenities' and 'description'.

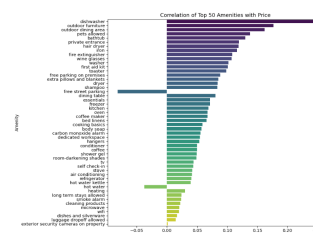


Figure 3. Correlation with Price

For the amenities feature, I begin by computing the correlation between the presence of each amenity category and the target variable, price. The top 15 amenity categories with the highest positive correlation are then selected for further use. Each of these selected amenities is transformed into a separate binary feature, indicating its presence (1) or absence (0) in a given listing.

To integrate textual data from the description field, TF-IDF vectorization is applied to extract the top 500 weighted terms while removing English stopwords. The high-dimensional TF-IDF matrix is then reduced to 50 components using Truncated SVD to capture key semantic patterns. These components are appended to the original dataset, enabling the model to leverage compressed textual features alongside structured inputs.

3.1.2. Handling Numerical Values

Following the processing of non-numeric features, the majority were successfully transformed into numerical representations. These numeric features are subsequently standardized using *StandardScaler* to ensure that they are on a comparable scale. This standardization step is essential for enabling effective application of dimensionality reduction techniques such as PCA, which are sensitive to feature magnitudes.

4. Dimensionality Reduction

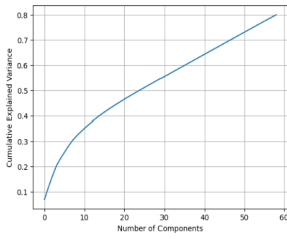


Figure 4. Capturing Variance

For the dimensionality reduction phase, Principal Component Analysis (PCA) is applied to transform the standardized feature space into a set of orthogonal principal components. The number of components is selected such that they collectively retain 80% of the total variance in the data. As illustrated in Figure 4, this threshold is achieved using 59 principal components, effectively reducing dimensionality while preserving the majority of the information content.

5. Modeling

5.1. Train Test Split

To prepare the data for model training and evaluation, the PCA-transformed dataset (*pca_train*) is split into training and validation subsets using an 80/20 ratio. This is accomplished using the *train_test_split* function from *scikit-learn*, with a *random_state* of 42 to ensure reproducibility. The resulting subsets—*X_train_sub*, *X_val_sub*, *y_train_sub*, and *y_val_sub*—are then used in subsequent modeling and validation steps.

5.2. Linear Regression

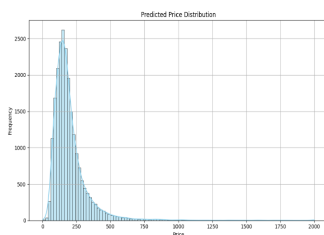


Figure 5. Linear Regression Prediction Distribution

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (1)$$

$$\hat{y} = \mathbf{x}^T \boldsymbol{\beta} \quad (2)$$

The linear regression model was fitted using the PCA-reduced training data to establish a baseline for performance. Post-prediction, all negative values were capped at zero to reflect the non-negative nature of the price variable. The model achieved a validation RMSE of 210.198 and MAE of 99.384, indicating reasonable error margins for a linear approach. The predicted price distribution remains right-skewed, with a mean of 199.82 and a maximum of 2,000, suggesting that while the model captures central trends, it struggles to model higher-end price variability due to its linear assumptions.

5.3. Ridge Regression

For Ridge Regression, *GridSearchCV* is employed to identify the optimal regularization parameter α , which controls the strength of the penalty applied to the model coefficients.

```
1 # Define parameter grid
2 param_grid = {
3     'alpha': [0.001, 0.01, 0.1, 1, 10, 100]
4 }
5 # Perform grid search with negative MSE scoring
6 ridge_grid = GridSearchCV(
7     Ridge(),
8     param_grid,
9     scoring='neg_mean_squared_error',
10    cv=5
11 )
12 ridge_grid.fit(X_train_sub, y_train_sub)
```

Code 1. Parameter Tuning

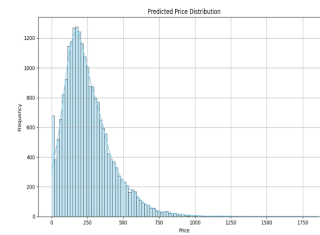


Figure 6. Ridge Regression Prediction Distribution

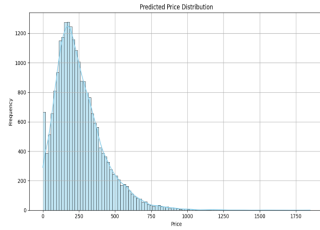
$$\mathcal{L}_{\text{ridge}}(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\beta})^2 + \alpha \|\boldsymbol{\beta}\|_2^2 \quad (3)$$

The Ridge Regression model yields a right-skewed predicted price distribution, with most values concentrated below \$500. Summary statistics report a mean of 253.87, and a maximum of 1,849.05. On the validation set, the model achieves an RMSE of 212.50 and MAE of 117.91, indicating comparable performance to the baseline linear model, with slight regularization helping to stabilize coefficient estimates.

5.4. Lasso Regression

For Lasso Regression, *GridSearchCV* is employed to identify the optimal regularization parameter α , which controls the strength of the penalty applied to the model coefficients.

```
1 param_grid = {'alpha': [0.0001, 0.001, 0.01, 0.1, 1, 10]}
2 ↵
3 lasso = Lasso(max_iter=10000)
4 grid = GridSearchCV(lasso, param_grid, scoring='
5 ↵ neg_mean_squared_error', cv=5)
6 grid.fit(X_train_sub, y_train_sub)
```

Code 2. Parameter Tuning**Figure 7.** Lasso Regression Prediction Distribution

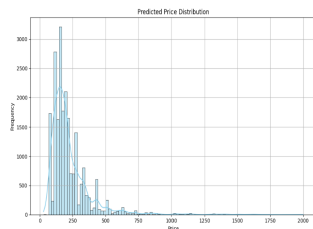
$$\mathcal{L}_{\text{lasso}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2 + \alpha \|\beta\|_1 \quad (4)$$

The Lasso Regression model produces a predicted price distribution that is right-skewed, closely resembling those of previous linear models. The predicted values have a mean of 253.9, and a maximum of 1,847.9. On the validation set, the model achieves an RMSE of 212.5 and MAE of 117.84, performing nearly identically to Ridge and Linear regression. This indicates that L1 regularization did not significantly affect predictive accuracy in this case, but may still contribute to feature sparsity.

5.5. Decision Tree Regressor

For the Decision Tree Regressor, GridSearchCV is used to identify the optimal combination of hyperparameters, including `max_depth`, `min_samples_split`, and `min_samples_leaf`. These parameters control the complexity of the tree and help mitigate overfitting. A 5-fold cross-validation strategy is applied with the objective of minimizing the mean squared error, resulting in the best configuration: `max_depth = 10`, `min_samples_split = 10`, and `min_samples_leaf = 4`.

```
1 param_grid = {
2     'max_depth': [5, 10, 20, None],
3     'min_samples_split': [2, 5, 10],
4     'min_samples_leaf': [1, 2, 4]
5 }
6 dt = DecisionTreeRegressor(random_state=42)
7 grid = GridSearchCV(dt, param_grid, scoring='
8     ↪ neg_mean_squared_error', cv=5)
9 grid.fit(X_train_sub, y_train_sub)
```

Code 3. Parameter Tuning**Figure 8.** Decision Tree Regressor Prediction Distribution

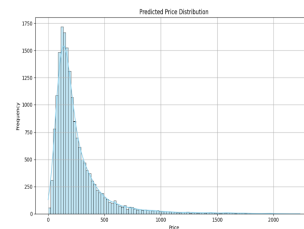
The Decision Tree Regressor yields a predicted price distribution that is highly segmented, reflecting the model's tendency to produce discrete output levels based on learned split thresholds. The predicted values exhibit a mean of 231.23, and a maximum of 2000, with several sharp peaks in the histogram. On the validation set, the model achieves an RMSE of 221.43 and an MAE of 114.6, showing similar performance over linear models. However, the spiky distribution

indicates potential overfitting and limited generalization due to the tree's non-smooth decision boundaries.

5.6. XGB Regressor

The XGBoost Regressor was optimized using RandomizedSearchCV over a defined hyperparameter space to improve predictive performance and reduce overfitting. A custom RMSE-based scoring function was used, with the objective set to minimize root mean squared error. The search spanned key hyperparameters including `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`, and `min_child_weight`. The model was evaluated using 3-fold cross-validation across 50 randomly sampled parameter combinations. The best-performing configuration, as determined by the lowest cross-validated RMSE, was selected and stored as the final XGBRegressor model for subsequent prediction tasks.

```
1 rmse_scorer = make_scorer(lambda y, y_pred: np.sqrt(
2     ↪ mean_squared_error(y, y_pred)), greater_is_better
3     ↪ =False)
4 # Define parameter grid
5 param_dist = {
6     'n_estimators': randint(100, 1000),
7     'learning_rate': uniform(0.01, 0.3),
8     'max_depth': randint(3, 10),
9     'subsample': uniform(0.6, 0.4),
10    'colsample_bytree': uniform(0.6, 0.4),
11    'reg_alpha': uniform(0, 1),
12    'reg_lambda': uniform(0, 1),
13    'min_child_weight': randint(1, 10)
14 }
15 # Base model
16 xgb = XGBRegressor(
17     objective='reg:squarederror',
18     random_state=42,
19     n_jobs=-1,
20     verbosity=1
21 )
22 # Random search
23 random_search = RandomizedSearchCV(
24     estimator=xgb,
25     param_distributions=param_dist,
26     scoring=rmse_scorer,
27     n_iter=50, # Number of random parameter
28     ↪ combinations to try
29     cv=3,
30     verbose=2,
31     random_state=42,
32     n_jobs=-1
33 )
34 # Fit on training data
35 random_search.fit(pca_train, y_train)
```

Code 4. Parameter Tuning**Figure 9.** XGB Regressor Prediction Distribution

The XGBoost Regressor produced a predicted price distribution that remains right-skewed but is smoother and more continuous compared to earlier models. The test predictions have a mean of 242.4, and a maximum of 1,943.19, indicating better spread and calibration across the price range. The model achieved an RMSE of 172.77 and an MAE of 88.16, significantly outperforming all previous models in terms of both accuracy and error reduction. This performance

highlights XGBoost's strength in capturing complex, nonlinear relationships in the data.

6. Result

Based on the evaluation metrics on the validation set, the XGBoost Regressor demonstrated the best performance among all models tested. It achieved the lowest error rates, with a Root Mean Squared Error (RMSE) of 172.77 and a Mean Absolute Error (MAE) of 88.16, outperforming both linear and tree-based models. This indicates that XGBoost effectively captures complex, nonlinear relationships in the data, leading to more accurate predictions.

Therefore, considering the lower RMSE and MAE values, the XGBoost Regressor stands out as the most effective model for predicting prices in this dataset.

For the final prediction on the test set, the model yielded an RMSE of 544.63. The significant gap between this test RMSE and the lower validation RMSE suggests overfitting, likely due to the model's complexity. To address this in future iterations, a simpler and more regularized model should be considered to improve generalization performance.